

Lampwarden asset pipeline

Pixel art as code. A model writes it through Mubit, then it is rendered, judged and fixed.

For Lampwarden, a night-defence roguelite in Lychnos, a Greek harbour town.

Built at the Mubit Night Shift hackathon, London, 24 October 2026.

lampwarden.vercel.app · github.com/frostney/lampwarden

The game



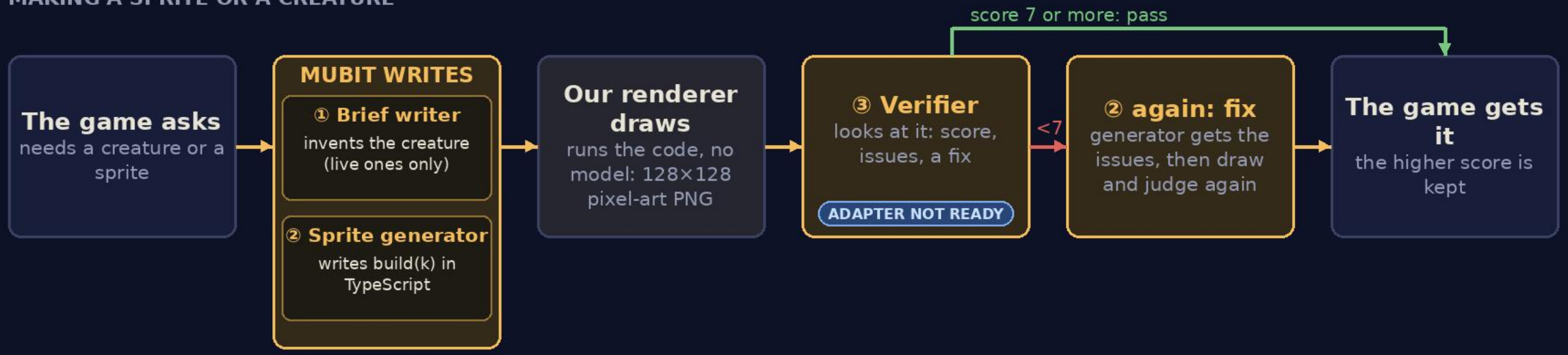
Night 1 on main at 4eddbd9: screenshot.png from commit 0a3e15e (PR #77), 25 Oct 2026, 01:15 UK.

- Each night the sea sends creatures to put out the town's lamps.
- Strike them down, relight the lamps, keep one burning until dawn.
- Dawn pays oil. After every night a shop offers three of nine upgrades.
- When a warden falls, an heir takes the lamp, with one of five traits.
- Keyboard and mouse, gamepad or touch.

Source: README.md, screenshot.png

Where Mubit comes in

MAKING A SPRITE OR A CREATURE



SETTING EACH NIGHT'S DIFFICULTY



4 Mubit calls, 4 prompt files: ① creature.md · ② generator.md (also the fix) · ③ verifier.md · ④ director.md
The model never draws a pixel or sets a number the game doesn't clamp.

Source: pipeline/loop.ts, pipeline/prompts/{creature,generator,verifier,director}.md, README.md. Verifier adapter: not ready.

Lampwarden · Mubit Night Shift, London

The isometric kit

THE PROJECTION (kit.ts header)

```
World: u runs screen down-right, v down-left, z up. Screen = (64
+ 2(u - v), 100 + u + v - sqrt6*z). 128x128 canvas.
```

Pure TypeScript, no imports, deterministic: own sin/cos/atan (only + - * / sqrt floor)
Bun versions and CPU architectures don't promise identical Math.sin results, so the sandbox and a local runtime would otherwise differ.

WHAT THE KIT DOES FOR THE MODEL (quoted)

- "iso projection, z-buffer, banded Lambert light, auto outline, cast shadow, 128x128" generator.md
- "banded lambert + ground bounce on undersides + lit rim + ordered dither on band seams" kit_render.ts
- "cast shadow on the ground plane z=0 (translucent), only on empty pixels" kit_render.ts
- "post step (no LLM): outline, palette clamp, bounds, coverage" kit_render.ts

HAND-WRITTEN EXAMPLE (not Mubit output): ex_house.ts

```
export function build(k: Kit) {
  house(k, { w: 24, d: 18, storeys: 1, wall:
      'whitewash', roof: 'flat', door: 'u',
      windows: 2, shutters: 'aegean', extras:
      ['pots', 'amphora', 'chimney',
      'bougainvillea'] });
}
```



house-handwritten.png

KIT EXAMPLE (not Mubit output): ex_hoplite.ts

```
export function build(k: Kit) {
  humanoid(k, {
    facing: 20, build: 'average', pose: 'idle',
        skin: 'skin',
    head: { type: 'human', helmet: 'corinthian',
        crest: 'crimson' },
    torso: { armour: 'linothorax', trim: 'aegean'
        }, legs: { greaves: true, sandals: true },
    weapon: 'spear', shield: { type: 'hoplon',
        colour: 'bronze', emblem: 'lambda',
        emblemColour: 'crimson' },
    cape: 'crimson',
  });
}
```



rendered from this file with the kit

The model never touches a pixel. It picks builders and parameters; the kit projects, shades and outlines them.

Source: pipeline/kit/kit.ts, kit_render.ts, pipeline/prompts/generator.md, kit/examples/ex_house.ts, ex_hoplite.ts, house-handwritten.png

Inside the kit

PROJECT: world (u, v, z) to a screen pixel (pipeline/kit/src/kit_core.ts)

```
102  export const LIGHT: V = norm([0.15, 0.5, 0.85]);
103  export const ZK = Math.sqrt(6);
104  export const VIEW: V = norm([0.5, 0.5, 1 / ZK]);
108  export function proj(p: V): [number, number] { return [OX + 2 * (p[0] - p[1]), OY + p[0] + p[1] - ZK * p[2]]; }
```

← u, v, z in, x, y out

SHADE: banded Lambert light on every pixel (pipeline/kit/src/kit_render.ts, indent of 8 removed)

```
207  const lam = Math.max(0, dot(n, LIGHT));
208  // form shading: banded lambert + ground bounce on undersides + lit rim + ordered dither on band seams (smooth, not noisy)
209  let kf = 1 + (NN - 2) * (0.12 + 0.88 * lam);
210  if (n[2] < 0) kf += Math.min(1, -n[2]) * 0.55 * (1 - lam);
211  const ndv = dot(n, VIEW); if (ndv < 0.38 && lam > 0.2) kf += (0.38 - ndv) * 1.4;
212  const kb = Math.floor(kf), fr = kf - kb; const bx = i % W, by = (i - bx) / W;
213  if (fr > 0.3 && fr < 0.7) k = kb + (((fr - 0.3) / 0.4) > BAYER[(by & 3) * 4 + (bx & 3)] / 16 ? 1 : 0); else k = Math.round(kf);
214  k += sh;
215  if ((m === 'bronze' || m === 'gold' || m === 'iron') && dot(n, HALF) > 0.965) k = NN - 1;
216  else if (m.charAt(0) === 's' && m.charAt(1) === 'k' && dot(n, HALF) > 0.985) k += 1;
217  else if (m === 'snake' && dot(n, HALF) > 0.97) k += 1;
```

PROJECT

Every shape lives in world space.

proj() puts a point on the 128×128 canvas.

SHADE

lam is how much the surface faces LIGHT. It sets the shade band k, and R[k] is the pixel colour.

Dither only on the band seams.

OUTLINE

Next step, no LLM (kit_render.ts:308): "outline, palette clamp, bounds, coverage".

Line 317: "outer outline: darkest shade of the front-most neighbouring part".

I keep this part out of the model's hands. It picks shapes and materials. This code decides every pixel.

Source: pipeline/kit/src/kit_core.ts:102-104,108 and kit_render.ts:207-217,308,317 (main at 3cd6a3a). Same text is in kit/kit.ts.

Pixel art is code

THE KIT API THE MODEL GETS (quoted from generator.md). IT WRITES ONLY build(k)

You write ONE short TypeScript function `build(k)` that composes a single isometric pixel-art sprite … A fixed renderer (iso projection, z-buffer, banded Lambert light, auto outline, cast shadow, 128x128) draws whatever you add to `k`. You only choose and parameterise builders.

ship(k, {at, facing:5 (0 side-on, bow right; …), kind:'trireme'|'galley', sail:true|false, oars:5, dock:true|false, setting})



Brief: "war galley moored at a wooden dock"

verifier: first try, score 7

```
function build(k) {
  ship(k, { at: [0, 0], facing: 5, kind: 'galley', sail: true, oars: 5, dock: true });
}
```



Brief: "Polyphemus the one-eyed Cyclops with a huge club"

verifier: best of 5 candidates, score 7

```
function build(k) {
  cyclops(k, { weapon: 'club' });
}
```



Brief: "ancient olive tree with a twisted, knotted trunk"

verifier: after the fix call, score 7

```
function build(k) {
  tree(k, { at: [0, 0], kind: 'olive', height: 30 });
  k.capsule([1, 1, 1.5], [5, 3, 0.3], 1.4, 0.7, 'leather');
  k.capsule([-1, 0.5, 1.2], [-4, 3, 0.3], 1.2, 0.6, 'leather', { shift: -1 });
}
```

Source: pipeline/prompts/generator.md, public/sprites/ (galley-dock, cyclops, olive), manifest.json

A correction in action: the Greek trireme



ATTEMPT 1
REJECTED

5

/ 10

verifier says no
(pass needs 7)



ATTEMPT 2
PASSED

7

/ 10

after the fix call
two verifications: 7, 7

Verifier issues (verbatim, shortened):

- Hull is foreshortened into a short, tall, basket-like tub …
- The bronze ram looks detached: a gold cob-shaped blob floats below and in front of the hull …
- The large sail and mast cover most of the hull …

Verifier's fix: "Turn the ship closer to side-on … Raise oars to 8 or more … Set sail: false …"

What the fix call changed (build(k) arguments):

```
facing:    20  →  3
sail:    true  →  false
oars:       5  →  9
```

Attempt 2 is the shipped sprite (public/sprites/trireme.png is byte-identical to the fixed render).

The verifier still finds flaws: "The oars bunch into a dense comb near the stern …"

```
a1_z05.ts   ship(k, { at: [0, 0], facing: 20, kind: 'trireme', sail: true, oars: 5, dock: false });
a1_z05b.ts  ship(k, { at: [0, 0], facing: 3, kind: 'trireme', sail: false, oars: 9, dock: false });
```

Source: rig/loop6/gen/a1_z05.ts, a1_z05b.ts, rows/a1_z05.json, 24 Oct 2026 11:09–11:10 UK

Lampwarden · Mubit Night Shift, London

One piece of code, a spritesheet with animation

THE WARDEN, WRITTEN BY THE MODEL (public/sprites/hero/male.ts)

```
function build(k) {
  hero(k, {
    sex: 'male',
    melee: 'xiphos',
    ranged: 'bow',
    pose: POSE || 'idle', ← the frame, idle1 … ranged5
    facing: FACING, ← the direction, in degrees
    helmet: 'none',
    cape: true,
    palette: { cloth: 'marble', accent: 'crimson', hair: 'fur_dark', skin: 'skin', metal: 'bronze' },
  });
}
```

THE LAYOUT (src/scene/assets.ts)

- 20 frames per sex: idle 4, walk 6, melee 5, ranged 5
- frames per second: idle 6, walk 10, melee 14, ranged 12
- 5 drawn directions: e, se, s, ne, n. The other three are mirrored.
- 128 px cells. male.png is 640×2560: 5 columns × 20 rows
- 100 renders per sex, all from this one build(k)

WALK, SOUTH COLUMN: walk1 … walk6, played at 10 fps



walk1 walk2 walk3 walk4 walk5 walk6

atlas rows 4 to 9, column 2

ONE WALK FRAME, FIVE DIRECTIONS (walk1, atlas row 4)



e se s ne n

frames go down the atlas, directions go across

Creatures: idle 4, walk 6, attack 5 (6, 10, 14 fps), rendered from the creature's own code in one sandbox batch. “a builder that ignores POSE draws the same figure in every frame, which is caught here and refused.”
bun run check:pixels: 0 differing pixels over all 218 renders.

I never draw a frame. The same code runs 100 times, with a different POSE and FACING each time.

Source: public/sprites/hero/male.ts and male.png, src/scene/assets.ts, pipeline/hero-animation.ts, hero-model.ts, creature-frames.ts, README.md

Lampwarden · Mubit Night Shift, London

Live creatures, one shared gallery

- Before each night the game announces "Tonight the sea sends…" and the newest creature the player hasn't met.
- One Mubit call writes the creature: name, lore, sprite brief, archetype and modifiers.
- Three candidates, each rendered in its own network-less Vercel Sandbox. The verifier judges each.
- Accepted creatures go into one shared gallery.
- Stats come from an archetype (skulker, brute or flyer) plus small modifiers, never model-written numbers.

Measured on 24 October: 8 of 10 jobs accepted, in about 40 s each. The night never waits for it.

Source: README.md, docs/LIVE_GENERATION.md

The difficulty director

IT READS (one JSON per night)

- the night just played: tide, outcome (dawn, dark, fallen, abandoned), kills a minute
- lamps lost and relit, health lost, how crowded it got
- was the player there: engaged, idle, paused or left
- a timeline every 5 s: [second, kills, lamps lit, health, monsters alive]
- its own last decision and whether it worked; what the rules would pick



IT RETURNS (JSON, nothing else)

```
{
  "reasoning": "≤ 20 words",
  "tide": 0.95,
  "mix": { "satyr": 1.0, "cyclops": 0.7,
           "harpy": 1.2 },
  "creatureShare": 0.25,
  "line": "≤ 90 characters"
}
```

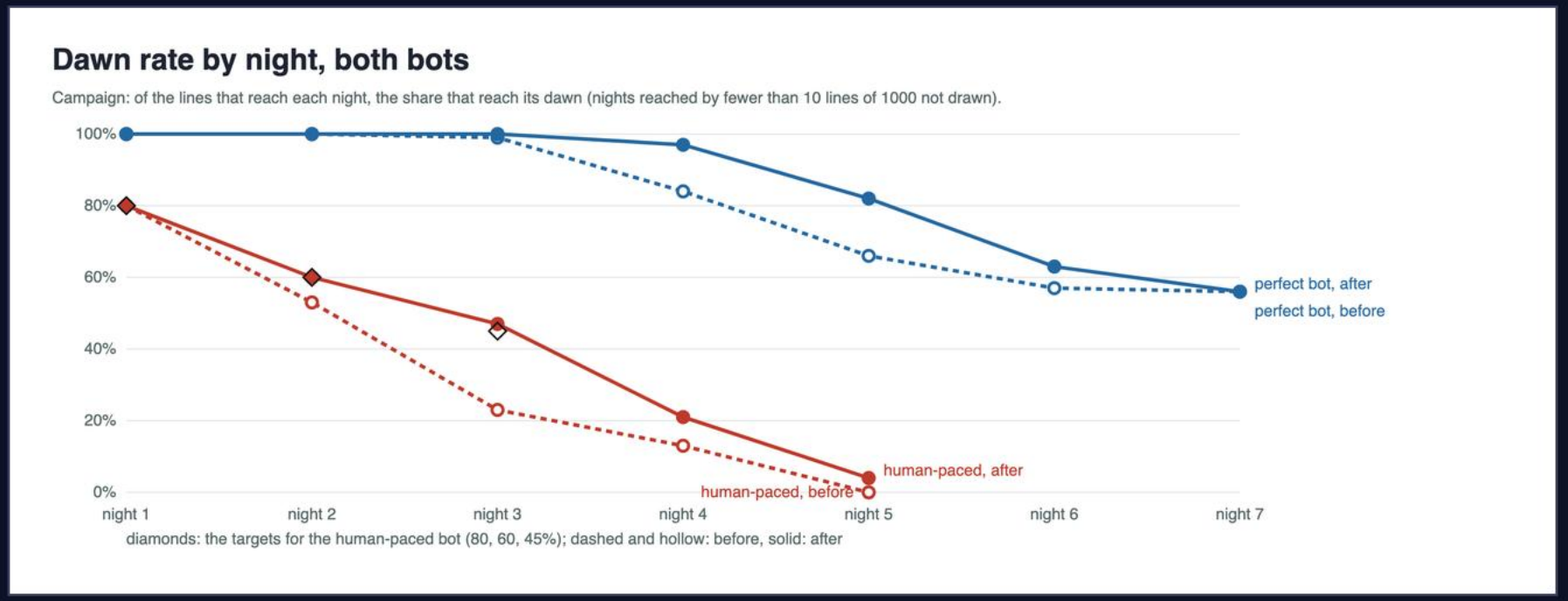
THE GAME HOLDS IT TO (in code):

- tide: within 0.15 of the rules' tide (0.6 to 2)
- mix: 0.5 to 2 for each kind of monster
- creatureShare: 0.1 to 0.4
- line: at most 90 characters, plain text

- Never trusted: the reply is clamped (clampProposal), then checked again (checkedDecision).
- After an idle, paused or left night it can only hold or push (PR #52): tide at least the rules', every mix weight at least 1, creature share at least 0.25.
- No key, timeout, error or bad JSON: the rules' decision plays instead. Nothing is asked of the player.

Source: director.md, director-decision.ts, director.ts, docs/DIRECTOR.md, PR #52

Tuned on people, not on a bot



PR #55 chart: dawn rate by night, human-paced and perfect bot, before (dashed) and after (solid).

WHY

- Nine recorded nights from three people: they lost night 2 every time (PR #47).
- Night 1 reached dawn on 3 of 5 plays, night 2 on 0 of 4 (PR #55).
- I had tuned the curve on a scripted bot that kills 24 to 31 a minute. I added a human-paced one.

WHAT CHANGED, in the PR's own numbers

Human-paced bot, dawn rate, nights 1 to 5: before 80 / 53 / 23 / 13 / 0 % → after 80 / 60 / 47 / 21 / 4 % (targets about 80 / 60 / 45).

Perfect bot: 82 % on night 5 and 63 % on night 6 (66 % and 57 % before). Spawn ramp over nights 1 to 3 is now 12 % a night (was 30 %).

Honest limit: the nine human nights' recordings are not in the repo. The chart is a simulated bot calibrated to them.

Source: PR #55, PR #47, scripts/human-bot.ts

The director, answering real nights

call	the night	tide played	kills · lit · health	next tide	mix satyr · cyclops · harpy	the sea says	why (the model's reasoning, logged, never shown to players)	model
play 5	night 2, dark	1.1	2 · 0 · 38	**0.79** (rules 0.94, -0.15)	1 · 0.6 · 0.7	*The sea draws back its hand, and the lamps of Lychnos may yet hold.*	Rules' night 2 at tide 1.1 went dark in 18s: all five lamps lost between 8s and 14s, none relit, six alive at 15s, 2 kills, health down to 38. The pace and the fast snuffers beat this player, so I ease fully and cut cyclops and harpies.	6.1 s
chain A step 2	night 2, dark	1.18	2 · 0 · 38	**0.85** (rules 1, -0.15)	1 · 0.6 · 0.6	*The sea lets go of the harbour stones, and Lychnos hears only the gulls.*	My push failed: at tide 1.18 all five lamps went dark by 18 s with none relit and six monsters alive at 15 s, while health was still 38. The pace and the lamp-snuffers beat the player, so I ease to the floor and cut cyclops and harpies.	6.5 s
chain A step 3	night 1, dawn	0.85	15 · 5 · 64	**0.93** (rules 0.85, +0.08)	1 · 0.8 · 0.9	*The swell stirs again beneath the harbour wall, and Lychnos feels it breathe.*	My easing worked: a dawn with all five lamps lit, both lost lamps relit, 22.5 kills a minute and health at 64. Night two broke this line before, so I push only gently and keep the cyclops and harpies a little below normal.	5.9 s

Three rows of the PR #47 table, cropped: real replies (play 5, then a chain where the model sees how its last decision went).

Chain A, step 2: night 2 went dark at tide 1.18

The model's reasoning (logged, not shown to the player):

"My push failed: at tide 1.18 all five lamps went dark by 18 s with none relit and six monsters alive at 15 s, while health was still 38. The pace and the lamp-snuffers beat the player, so I ease to the floor and cut cyclops and harpies."

The sea says (shown on the dawn / heir screen):

The sea lets go of the harbour stones, and Lychnos hears only the gulls.

next tide 0.85 (rules 1, -0.15)

Chain A, step 3: the eased night 1 reached dawn

The model's reasoning (logged, not shown to the player):

"My easing worked: a dawn with all five lamps lit, both lost lamps relit, 22.5 kills a minute and health at 64. Night two broke this line before, so I push only gently and keep the cyclops and harpies a little below normal."

The sea says (shown on the dawn / heir screen):

The swell stirs again beneath the harbour wall, and Lychnos feels it breathe.

next tide 0.93 (rules 0.85, +0.08)

Text typed from the PR image. These use the first answer shape; the shipped default (V2) answers in one short sentence (PR #47).

Source: PR #47, image "The live director, against Mubit" (pr47-live-decisions-full.png)

What Mubit gives us

- **One gateway for every model call.** Brief writer, generator, verifier, fix and director all go to api.mubit.ai on claude-opus-5-5. Load check: 24 generator calls, all 200, no retries.
- **A request id and a session on every call.** The manifest keeps them. The trireme's row lists its generator, verifier, fix and two re-verify ids, so each step can be looked up in the Mubit console.
- **Calls grouped into tasks.** Same model and system prompt, same task. Mubit keeps examples and can train a small adapter per task, tested against the provider model's answers (docs.mubit.ai).
- **Caching and limits pass through.** The generator prompt (5,379 tokens) was cached once and read back by the next call. Anthropic's rate-limit headers pass through (1,000 requests/min, 2M input tokens/min).

The verifier is its own Mubit task. The adapter is not ready, so verifier calls still run on claude-opus-5-5.

The correction already works in-context: issues and fix go back to the fix call as a follow-up turn.

My code doesn't read the console's Good/Bad rating (pipeline/README.md).

Source: pipeline/mubit.ts, pipeline/README.md, docs/LIVE_GENERATION.md, docs.mubit.ai

Play it

lampwarden.vercel.app

github.com/frostney/lampwarden

```
bun install && bun run dev
```

